Tabla de contenidos

1. WebServices
2. API (*Application Programming Interface*)
3. URL (*Uniform Resource Locator*)
4. HTTP (*Hypertext Transfer Protocol*)
5. TLS (*Transport Layer Security*)(BONUS)

WebServices

En nuestro primer acercamiento al uso de networking, revisaremos la comunicación entre dos dispositivos mediante la **web**.

Web services es el conjunto de aplicaciones cliente-servidor que se comunican a través de la web mediante un protocolo diseñado para ello. Podemos ver este tipo de servicios como aplicaciones que pueden ser accedidas por otras aplicaciones a través de una red de computadores (internet).

Por ejemplo, cuando nuestro navegador (cliente) consume un sitio web (servidor), por cada **llamada al servidor**, una aplicación escrita en algún lenguaje de programación **envía una respuesta** en HTML (el lenguaje que se utiliza para definir la estructura de un sitio web) para que nuestro navegador la despliegue. Los *web services* funcionan de forma similar, donde la salida está dirigida a una **aplicación que consume** esta información. Para que la comunicación sea posible, el formato de los mensajes debe ser conocido por ambas partes, para que la información pueda ser interpretada correctamente.

API (*Application Programming Interface*)

En general se conoce como **API** al conjunto de funciones que son expuestas por un servicio para ser utilizadas por otros programas. Podemos ver al servicio como una clase, y a la API como el conjunto de métodos de esa clase. El servicio puede ser un *web service* o cualquier paquete que exponga una interfaz, por ejemplo una librería de Python. Sin embargo, en este capítulo cuando hablemos de API nos estaremos refiriendo a los métodos expuestos por un servicio web (un *web service*).

Una gran parte de los servicios actuales exponen una API, y permiten que otras aplicaciones se conectan a ellas. De esta manera podemos construir aplicaciones que utilizan servicios que se encuentren en computadores remotos, e interactuar con ellos.

En una red de computadores, cada página web de internet es almacenada en un computador remoto que ejecuta un proceso servidor. Un servidor remoto es simplemente un programa que escuchas *requests* (solicitudes) y envía *responses* (respuestas) de acuerdo a un protocolo.

Tú puedes utilizar tu propio computador para servir un sitio web. De hecho, los desarrolladores de *software* usan sus propios computadores como servidores locales al crear sitios web antes de publicarlos al mundo.

Cuando escribes https://www.spotify.com en tu navegador, este envía una *request* a un servidor remoto de Spotify. Una vez que tu navegador recibe la respuesta del servidor, este la interpreta y despliega una página para ti.

Para el navegador (cliente), el servidor de Spotify es una API. Esto significa que cada vez que tú visitas una página en la web, tú interactúas con alguna API en un servidor remoto. Una API no es lo mismo que un servidor remoto, pero es la parte de este que recibe las *requests* y envía *responses* (respuestas).

Si cada objeto tecnológico de tu casa expusiera una API, podrías controlarla completamente desde tu celular o desde cualquier programa en Python u otro lenguaje (ver más en internet de las cosas).

URL (*Uniform Resource Locator*)

Una **URL** es la dirección que identifica de forma única un recurso en la web. Toda request HTTP comienza por construir o conocer la URL del recurso al que se quiere acceder.

Estructura de una URL

```
https://api.ejemplo.com:8080/users/42/posts?
page=2&limit=10#comentarios
└──┘   └──────────┘ └──┘ └──────────┘ └──────────┘ └──────┘
 esquema      host       puerto    ruta         parámetros
fragmento
```

Componente	Ejemplo	Descripción
Esquema (*scheme*)	`https`	Protocolo a usar (`http`, `https`, `ftp`, …).
Host	`api.ejemplo.com`	Nombre de dominio o dirección IP del servidor.
Puerto	`8080`	Puerto TCP. Opcional: si se omite se usa el puerto por defecto del esquema (`80` para HTTP, `443` para HTTPS).
Ruta (*path*)	`/users/42/posts`	Identifica el recurso dentro del servidor.
Query string	`page=2&limit=10`	Parámetros clave-valor que modifican la solicitud. Van después de `?`, separados por `&`.
Fragmento (*fragment*)	`#comentarios`	Sección dentro del documento. Solo lo usa el navegador; **no se envía al servidor**.

HTTP (*Hypertext Transfer Protocol*)

Gran parte de las arquitecturas de *web services* se basan en el uso del protocolo **HTTP**. Este protocolo de aplicación está encargado de proporcionar una capa para realización de

transacciones y así permitir la comunicación entre clientes y servidores. HTTP trabaja como un protocolo ***request-response*** en donde el cliente hace una solicitud (*request*) y el servidor responde con la información solicitada (*response*).

HTTP es un protocolo en el que el servidor no guarda ninguna información de las conexiones. Por ejemplo, al acceder a métodos de un servicio web que requiere identificación del cliente, este deberá en cada consulta enviar **algo** que acredite su identidad.

El funcionamiento de este protocolo se basa en la definición de **métodos o verbos** que indican la acción a desarrollar por un determinado recurso. Los recursos pueden ser datos existentes en el servidor (por ejemplo, archivos) o bien una salida generada dinámicamente. La versión HTTP/1.1 (RFC 9112) define **nueve** métodos:

- `GET`: recupera una representación de un recurso sin cambiar nada en el servidor.
- `HEAD`: igual que `GET`, pero el servidor solo devuelve los *headers*, sin *body*. Útil para verificar si un recurso existe.
- `POST`: crea un recurso.
- `PATCH`: aplica modificaciones parciales a un recurso.
- `PUT`: reemplaza completamente un recurso existente.
- `DELETE`: elimina un recurso.
- `OPTIONS`: consulta qué métodos soporta el servidor para un recurso determinado.
- `CONNECT`: establece un túnel hacia el servidor (usado por proxies HTTPS).
- `TRACE`: realiza una prueba de bucle (*loopback*) a lo largo de la ruta hacia el servidor (principalmente para diagnóstico).

Sin embargo, los métodos más utilizados son GET, POST, PATCH, PUT, DELETE.

Códigos de estado HTTP

HTTP incluye un conjunto de **códigos de estado** mediante los cuales se entrega información al cliente sobre el resultado de su petición. Los códigos están agrupados en cinco categorías según su primer dígito:

Rango	Categoría	Significado general
1xx	Informacional	La solicitud fue recibida y el proceso continúa
2xx	Éxito	La solicitud fue recibida, entendida y aceptada con éxito
3xx	Redirección	Se requiere acción adicional para completar la solicitud
4xx	Error del cliente	La solicitud contiene errores o no puede ser completada
5xx	Error del servidor	El servidor falló al intentar completar la solicitud

Algunos códigos comunes de respuesta son:

2xx — Éxito

- (*)`200 OK`: solicitud exitosa. Es la respuesta estándar para peticiones `GET`.
- (*)`201 Created`: recurso creado exitosamente. Típico de peticiones `POST`.

- `204 No Content`: solicitud exitosa pero sin contenido en la respuesta. Típico de `DELETE`.

3xx — Redirección

- `301 Moved Permanently`: el recurso se ha movido de forma permanente a una nueva URL.
- `304 Not Modified`: el recurso no ha cambiado; el cliente puede usar su versión en caché.

4xx — Error del cliente

- `400 Bad Request`: la solicitud está mal formada o contiene parámetros inválidos.
- (*)`401 Unauthorized`: se requiere autenticación; las credenciales no fueron proporcionadas o son inválidas.
- (*)`403 Forbidden`: la petición es válida, pero el servidor rechaza responderla (sin permisos).
- (*)`404 Not Found`: el recurso solicitado no existe en el servidor.
- `405 Method Not Allowed`: el método HTTP usado no está permitido para ese recurso.
- `429 Too Many Requests`: el cliente ha enviado demasiadas solicitudes en poco tiempo (*rate limiting*).

5xx — Error del servidor

- (*)`500 Internal Server Error`: error genérico; algo falló dentro del servidor.
- `502 Bad Gateway`: el servidor actuando como *proxy* recibió una respuesta inválida.
- `503 Service Unavailable`: el servidor no está disponible temporalmente (sobrecarga o mantenimiento).

En el mundo real, los códigos con (*) son los mas comunes. Para más detalle de los códigos pueden revisar el siguiente enlace.

Adicionalmente, tanto las *request* como las *response*:

- Presentan un ***header***, el cual almacena información adicional a la consulta o respuesta, como es el caso de las credenciales de autentificación, almacenamiento en cache, cookies, entre otros.
- Pueden presentar un **body**, el cual corresponde a los datos transmitidos en la consulta o respuesta.
  El tipo de información que almacena el *body* de un mensaje puede variar, por lo que se utiliza el *header* `Content-type` para indicar el tipo de información que almacena el *body*. Algunos de estos son:
  - HTML (`Content-Type: text/html`)
  - JSON (`Content-Type: application/json`)
  - Texto plano (`Content-Type: text/plain`)
  - Imágenes (`Content-Type: image/jpeg, image/png, image/gif` dependiendo del tipo de archivo)

Para resumir todo lo anterior, la siguiente figura muestra un ejemplo con la estructura de los mensaje HTTP para la *request* del cliente y para la *response* desde el servidor.

=========BONUS=========

TLS (*Transport Layer Security*)

¿Qué es TLS?

TLS es el protocolo estándar para **cifrar las comunicaciones** en internet. Cuando una URL comienza con `https://` en lugar de `http://`, la conexión está protegida por TLS. Es el sucesor de SSL (*Secure Sockets Layer*), aunque ambos nombres se siguen usando indistintamente en la práctica.

TLS actúa **entre la capa de aplicación y la capa de transporte**, por encima de TCP pero por debajo de HTTP, y es completamente **transparente** para la aplicación: el código que hace una petición HTTP no necesita cambiar para beneficiarse de TLS.

```
Capa de Aplicación  |  HTTP, FTP, SMTP, ...
--------------------|-----------------------
Capa de Seguridad   |  TLS / SSL  ◄── cifrado aquí
--------------------|-----------------------
Capa de Transporte  |  TCP
--------------------|-----------------------
Capa de Red         |  IP
```

¿Qué protege TLS?

TLS garantiza tres propiedades fundamentales en la comunicación:

Propiedad	Descripción
Confidencialidad	Los datos viajan cifrados; nadie que intercepte el tráfico puede leerlos.
Integridad	Cualquier modificación de los datos en tránsito es detectada y la conexión se rechaza.
Autenticación	El cliente puede verificar que realmente está hablando con el servidor correcto (no un impostor).

El *handshake* TLS

Antes de intercambiar datos, cliente y servidor realizan un **TLS handshake** (negociación) para establecer la conexión segura. Los pasos principales son:

1. **ClientHello**: el cliente indica las versiones de TLS y los algoritmos de cifrado que soporta.
2. **ServerHello**: el servidor elige la versión y el algoritmo a usar.

3. **Certificado**: el servidor envía su **certificado digital** (emitido por una Autoridad Certificadora, o CA) para que el cliente verifique su identidad.
4. **Intercambio de claves**: cliente y servidor negocian una **clave de sesión** compartida usando criptografía asimétrica (por ejemplo, con el algoritmo Diffie-Hellman).
5. **Finished**: ambas partes confirman que el canal seguro está establecido y comienzan a enviar datos cifrados con la clave de sesión.

Certificados digitales y HTTPS

Un **certificado digital** es un documento electrónico que asocia una clave pública con la identidad de su propietario (por ejemplo, el dominio `www.banco.cl`). Lo emite y firma una **Autoridad Certificadora (CA)** de confianza (como Let's Encrypt, DigiCert o Comodo).

Cuando tu navegador visita `https://www.ejemplo.com`:

1. Recibe el certificado del servidor.
2. Verifica que esté firmado por una CA de confianza instalada en el sistema operativo.
3. Verifica que el dominio del certificado coincida con la URL solicitada.
4. Si todo es correcto, muestra el candado 🔒 en la barra de direcciones.

HTTP vs HTTPS

Característica	HTTP	HTTPS (HTTP + TLS)
Cifrado	No	Sí
Puerto por defecto	80	443
Integridad de datos	No garantizada	Garantizada
Autenticación del servidor	No	Sí (mediante certificado)
Velocidad	Más rápido (sin overhead)	Mínimo overhead con TLS 1.3
Recomendado para	Desarrollo local	Cualquier uso en producción